



**POLITECNICO DI BARI**

DEPARTMENT OF ELECTRICAL AND INFORMATION  
ENGINEERING

**Master's Degree in Automation Engineering**

---

Robotics II Project Work

**Differential drive follower robot**

Professor:

**Luca De Cicco  
Nunzio Barone**

Candidates:

**Lorenzo Frari**

---

ACADEMIC YEAR 2025–2026



# ABSTRACT

The author would like to emphasize that this project was not developed as a university assignment, but rather as a practical platform for implementing and integrating many of the concepts studied in robotics, computer science, electronics, and mechanics.

For this reason, the project required a significant amount of time, effort, patience, and dedication. It also includes several features that may not be strictly necessary for its primary objectives, such as local localization, safety stop mechanisms, frontier-based exploration, and teleoperation. These features were implemented to provide a broader learning experience and to explore a wider range of robotics-related topics.

As a consequence, the project is still a work in progress. There is considerable room for improvement in the existing features, and additional functionalities may be introduced in the future. For the sake of brevity, this report focuses exclusively on the aspects most closely related to robotics.



# Contents

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Problem Formulation</b>            | <b>1</b>  |
| <b>2</b> | <b>Modelling assumptions</b>          | <b>3</b>  |
| <b>3</b> | <b>Algorithmic choices</b>            | <b>5</b>  |
| 3.1      | Input . . . . .                       | 5         |
| 3.2      | Controller . . . . .                  | 5         |
| 3.3      | Local localization . . . . .          | 6         |
| 3.4      | Global localization . . . . .         | 6         |
| 3.5      | SLAM . . . . .                        | 7         |
| 3.6      | Navigation . . . . .                  | 7         |
| 3.7      | Follower . . . . .                    | 9         |
| 3.8      | Safety stop . . . . .                 | 10        |
| 3.9      | Real Hardware Interface . . . . .     | 10        |
| <b>4</b> | <b>Experimental Validation</b>        | <b>13</b> |
| 4.1      | Localization . . . . .                | 13        |
| 4.1.1    | Local localization . . . . .          | 13        |
| 4.1.2    | Global Localization . . . . .         | 15        |
| 4.2      | SLAM and target-following . . . . .   | 15        |
| <b>5</b> | <b>Critical discussion of results</b> | <b>17</b> |



# Chapter 1

## Problem Formulation

The objective of this project is to design and implement an autonomous differential-drive mobile robot capable of navigating in indoor environments while tracking a dynamic target.

The robot operates in two different scenarios. In the first one, the environment is unknown and the robot is required to build a map while simultaneously estimating its own pose using a Simultaneous Localization and Mapping (SLAM) algorithm. In the second scenario, a previously generated occupancy grid is available and the robot localizes itself within the map using a global localization algorithm.

Unlike the static obstacles represented in the occupancy grid, the target to be followed is a dynamic object that is not included in the map. Consequently, the robot must continuously detect the target using onboard sensors, estimate its relative position, and generate appropriate velocity commands while simultaneously avoiding collisions with both static and dynamic obstacles.

The mobile platform follows differential-drive kinematics. Therefore, all motion commands must satisfy the kinematic constraints of a non-holonomic robot. The navigation system is responsible for generating feasible trajectories towards the moving target while ensuring smooth and collision-free motion.

The overall objective can therefore be summarized as follows:

- build a map of an unknown environment or localize within a previously generated map;
- detect and estimate the position of a dynamic target;
- continuously navigate towards the target while respecting the robot kinematic constraints;
- safely avoid both static and dynamic obstacles during navigation.
- the robot must autonomously detect, track and follow the dynamic obstacle.





## Chapter 2

# Modelling assumptions

The following assumptions are considered throughout this work:

- The robot is modeled as a differential-drive mobile robot operating on a planar surface.
- Odometry is afflicted by error.
- The environment is represented by a two-dimensional occupancy grid containing only static obstacles (only during SLAM).
- The target to be followed is not represented in the occupancy grid and may move arbitrarily within the limits of the environment.
- Whenever the target lies within 3 meters from the sensor, its relative position can be estimated (this is due to noise reduction into estimating the new object).
- Robot localization is assumed to be sufficiently accurate for navigation purposes.
- Sensor measurements are affected by bounded noise.
- Regarding navigation purposes, robot's size is to be considered as a particle and not a body.
- Not all obstacles can be detected by the sensor.
- It's impossible for the robot to distinguish a person from an object.



## Chapter 3

# Algorithmic choices

The software architecture was designed with the objective of relying on well-established and modular algorithms available in the ROS 2 ecosystem whenever possible. This approach reduces development time while increasing reliability, allowing the implementation effort to focus on the integration of the various software components rather than on reimplementing existing solutions.

### 3.1 Input

The robot can receive velocity commands from different sources:

- Keyboard.
- Joystick.
- Autonomous navigation system.

As a consequence, the robot can operate either autonomously or under manual teleoperation. To correctly manage multiple command sources, the `twist_mux` node is used. Its purpose is to select, according to a predefined priority scheme, which velocity command is forwarded to the robot controller. This guarantees that only one source controls the robot at any given time while allowing seamless switching between autonomous navigation and manual control.

### 3.2 Controller

The robot control architecture is based on the `ros2_control` framework, which provides a hardware-independent interface between the high-level software and the robot actuators. The controller receives a desired linear velocity along the  $x$  axis and an angular velocity around the  $z$  axis and computes the angular velocity reference for each wheel according to the differential-drive inverse kinematic model.

Two controllers are available:

- `simple_velocity_controller`;
- `bumperbot_controller`.

The recommended choice is `bumperbot_controller`, which is implemented using the `controller_manager` package. Unlike the simple controller, it considers additional constraints such as velocity limits, acceleration limits, and jerk limits, resulting in smoother and safer robot motion.

On the other hand, `simple_velocity_controller` directly applies the inverse kinematic equations to convert the commanded linear and angular velocities into wheel angular velocities without considering any dynamic constraints. Although this controller is useful for understanding the underlying kinematics, it is generally not suitable for real-world applications.

The controllers generate an odometry estimate which, in simulation, is almost ideal, as it accurately represents the robot's true pose and velocity. In real robotic platforms, however, odometry is inevitably affected by measurement errors and mechanical imperfections. To better reproduce this behavior in simulation, a custom node named `noisy_controller` subscribes to the `joint_states` topic and artificially introduces small random errors into the wheel radius and wheel separation parameters used for odometry computation. These perturbations model real-world phenomena such as wheel wear, manufacturing tolerances, and slight wheel deformations. Although the errors introduced at each time step are very small, they accumulate over time, producing the drift that is typically observed in real differential-drive robots.

### 3.3 Local localization

Despite still being a work-in-progress feature, it is worth briefly discussing it. To mitigate the drift introduced by wheel odometry, an Extended Kalman Filter (EKF) has been implemented to fuse wheel encoder odometry with IMU measurements. The objective of this sensor fusion is to obtain a more accurate and robust estimate of the robot's state than could be achieved using either sensor independently.

### 3.4 Global localization

When a map is already available, the robot performs global localization using AMCL, which estimates the robot's pose by combining odometry information with laser measurements acquired from the LiDAR. By default, the algorithm initializes the robot's pose at the origin of the map. Therefore, in most cases there is no need to manually set the initial pose when the simulation starts (except when using the `small_warehouse` map).

### 3.5 SLAM

Regarding localization and mapping, the selected algorithms depend on the operating scenario. When the environment is unknown, the robot relies on SLAM to simultaneously estimate its pose and build a two-dimensional occupancy grid. The algorithm is implemented with the help of the `slam_toolbox`.

In the current state of the project a frontier detection algorithm is being developed.

### 3.6 Navigation

The navigation system is based on the ROS 2 Navigation Stack (Nav2). This framework provides all the components required for autonomous navigation, including path planning, path smoothing, obstacle avoidance, and recovery behaviors, allowing the robot to safely navigate cluttered indoor environments without developing a navigation framework from scratch. The navigation stack is composed of the following nodes:

- `nav2_bt_navigator`;
- `nav2_planner`;
- `nav2_smoother`;
- `nav2_controller`;
- `nav2_behaviors`.

When a navigation goal is received by `nav2_bt_navigator`, the corresponding Behavior Tree, designed using Groot2, is executed.

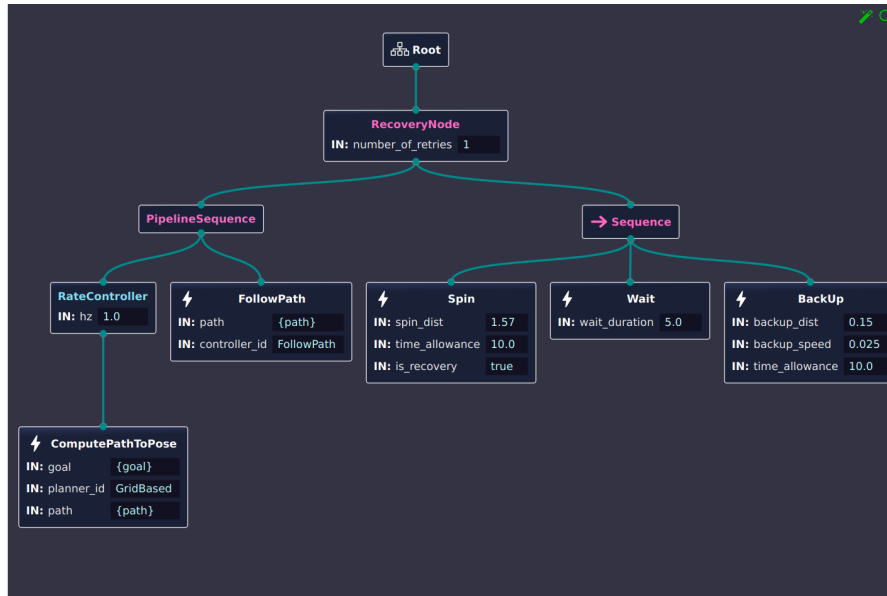


Figure 3.1: Behavior tree structure

The Behavior Tree coordinates the interaction between the different Nav2 components by continuously requesting a path from the planner. Once the path has been generated and smoothed, it is forwarded to the controller, which computes the velocity commands required to follow it. As the robot moves, this process is repeated continuously until the goal is reached.

The planner relies on the smoother server to transform the generated path into a smoother and more feasible trajectory before it is passed to the controller. If either the planner or the controller reports a failure, the Behavior Tree executes a recovery behavior by invoking `nav2_behaviors`. The configured recovery sequence consists of rotating the robot by approximately  $90^\circ$ , waiting for five seconds, and finally moving the robot backwards by a few centimeters before attempting to continue the navigation task.

The planner generates trajectories using the global costmap of the environment, whereas the controller operates on a local costmap covering an area of approximately  $3\text{m} \times 3\text{m}$  centered on the robot.

The global costmap is configured with the static, obstacle, and inflation layers. The static layer represents the occupancy grid, while the obstacle layer incorporates measurements from the LiDAR. Finally, the inflation layer enlarges the occupied regions by the robot radius and an additional configurable safety margin. This allows the planner to model the robot as a point while still generating collision-free trajectories. The local costmap uses only the obstacle and inflation layer but it's updated at a higher frequency.

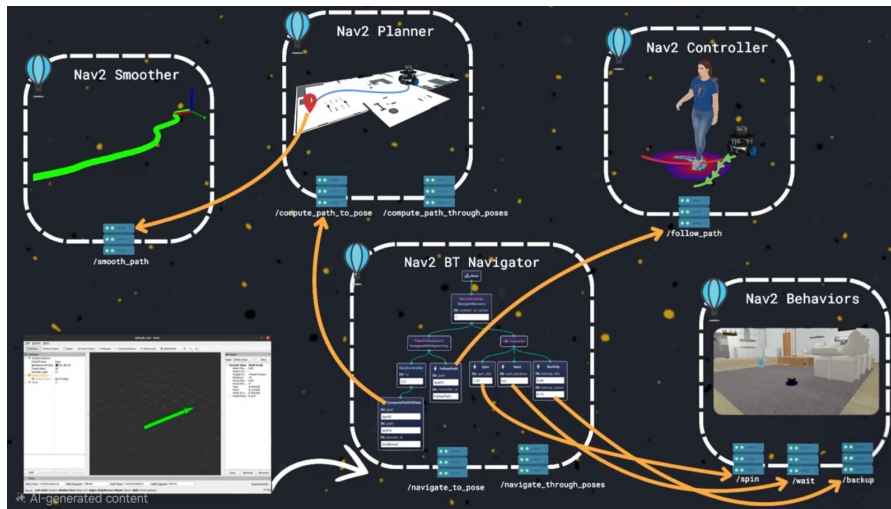


Figure 3.2: Navigation system workflow

### 3.7 Follower

The target-following problem was addressed by separating perception from navigation. A dedicated perception node detects the dynamic target using LiDAR data and continuously estimates its position. This information is then converted into navigation goals that are executed by the navigation stack. Such a modular architecture simplifies maintenance and allows each component to be replaced independently.

The perception node considers only LiDAR measurements within a radius of 3 meters from the sensor. Each measurement is converted into the corresponding cell of the occupancy grid and if that cell is marked as occupied the measurement is discarded. For every detected obstacle, a square area of  $5 \times 5 \text{ cm}^2$  centered on the corresponding cell is inspected to determine whether the obstacle is already represented in the occupancy grid.

If no known obstacle is found within this area, the measurement is considered to belong to a potential dynamic obstacle. Consecutive measurements that are closer than a predefined distance threshold are grouped together into a single cluster representing the same obstacle. Clusters composed of fewer than five measurements are discarded in order to reduce the effect of sensor noise.

For each remaining cluster, its centroid is computed. The distance between the robot and the centroid is then evaluated, and an offset is applied to estimate a suitable navigation goal located near the detected obstacle. If multiple valid clusters are detected, only the closest one is selected and its corresponding goal is forwarded to the navigation stack.

To reduce the number of navigation goals generated by noisy sensor measurements, a new goal is computed for a detected cluster only if its centroid has moved by approximately 30 cm since the previous goal was generated.

A limitation of the current implementation is related to the update mechanism of the navigation costmaps. Since the navigation stack plans paths using the costmaps, and these are updated only when the robot moves, a moving obstacle may leave behind occupied cells that are not immediately cleared. In this situation, the LiDAR correctly detects the new position of the cluster and a new navigation goal is generated. However, because the costmap still marks the previous position of the obstacle as occupied, the planner may fail to generate a valid path and the navigation stack triggers a recovery behavior. Once the recovery behavior causes the robot to move, the costmap is updated, the stale obstacle information is removed, and the robot can successfully plan a path toward the newly generated goal.

### 3.8 Safety stop

An obstacle avoidance mechanism has been implemented and is fully functional. The algorithm continuously monitors the distance to nearby obstacles and reduces the robot's speed whenever an object is detected within 60 cm. If the obstacle approaches closer than 20 cm, the robot is brought to a complete stop, preventing potential collisions.

This node interacts directly with the `twist_mux` node. To reduce the robot's speed, it publishes a scaling factor to the `turbo` topic, which scales only the velocity commands generated by the joystick. To stop the robot completely, the node disables the corresponding input in `twist_mux`, preventing any velocity command from being forwarded to the controller.

The following figure illustrates the implemented logic.

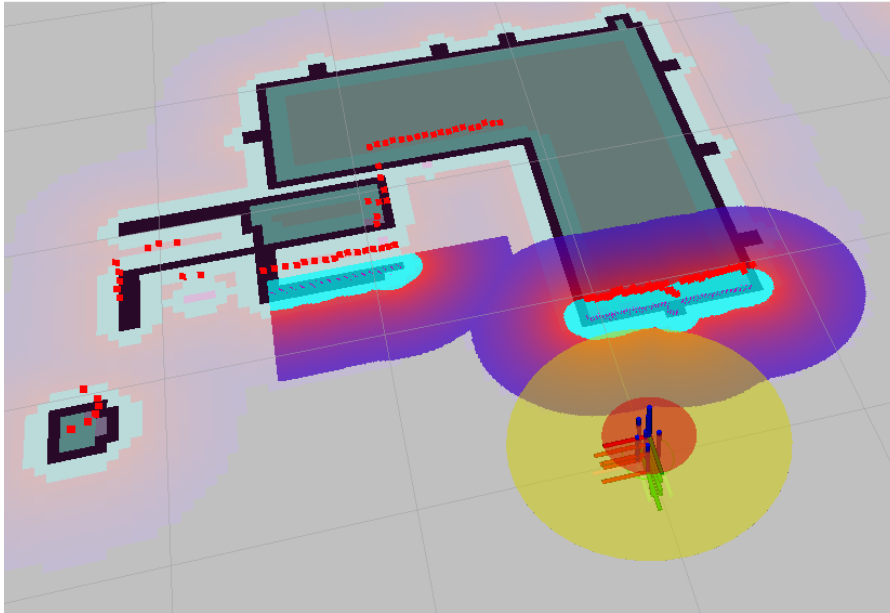


Figure 3.3: Safety zones

### 3.9 Real Hardware Interface

The robot control architecture is built upon the `ros2_control` framework. This framework provides a hardware-independent abstraction layer, allowing exactly the same control architecture to be used both in simulation and on the physical robot. As a result, the transition from simulation to the real platform requires only minimal software modifications.

A custom hardware interface has been developed to directly update the position and velocity state interfaces of the robot joints. These values are obtained from the wheel



encoder measurements processed by the microcontroller.

Similarly, the hardware interface forwards the wheel velocity commands generated by the controller to the microcontroller. Two independent PID controllers, one for each wheel, are executed on the microcontroller to minimize the error between the measured wheel velocity and the commanded reference.



## Chapter 4

# Experimental Validation

This chapter presents the experimental validation of the proposed system. The robot is evaluated both in simulation and on the real hardware to assess the performance of the implemented localization, navigation, and target-following algorithms. In this chapter only the results coming from the simulation are shown, the real one are provided only through the videos in the repository. The obtained results highlight the capabilities of the system.

### 4.1 Localization

#### 4.1.1 Local localization

The following figure illustrates the effect of the Extended Kalman Filter (EKF) in reducing the drift affecting the odometry estimate. The drift is caused by the artificial errors introduced into the wheel radius and wheel separation parameters, which simulate the mechanical imperfections typically found in real robotic platforms.



Figure 4.1: Local localization

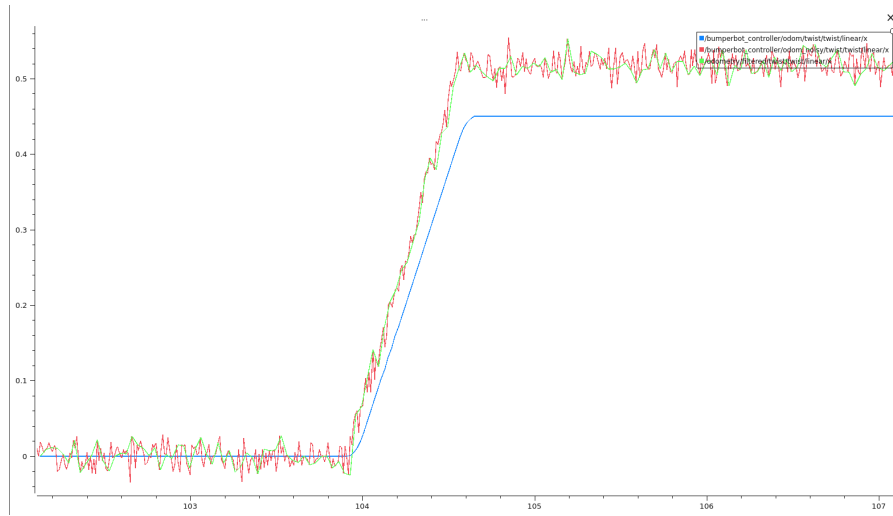


Figure 4.2: Local localization: velocity on x axes

The last picture shows:

- in blue the actual velocity of the robot(unknown in real world scenario).
- in red the velocity of the robot + measurement error.
- in green the velocity calculated from the ekf node.

It may seem that the ekf is not properly working but as seen in fig 5.1 the estimated pose calculated directly from the controller deviates very quickly from the real one as it accumulates errors over time, while the one calculated from the ekf is more reliable and most importantly it has some robustness to errors measurements.

### 4.1.2 Global Localization

The effect of the global localization algorithm is shown in the next figure. As can be observed, the AMCL algorithm continuously estimates the robot pose and publishes the transformation between the `map` and `odom` frames. By updating this transformation according to the estimated pose of `base_footprint`, the accumulated odometry drift is compensated, allowing the robot to maintain an accurate global pose estimate.

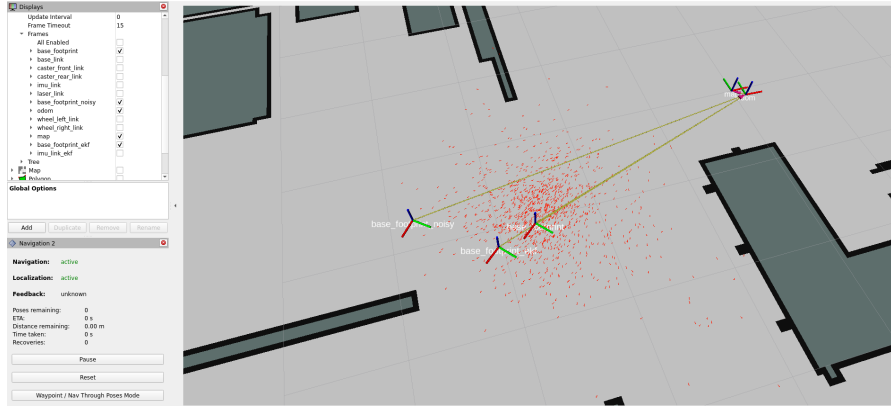


Figure 4.3: Global localization

## 4.2 SLAM and target-following

Since the quality of these features cannot be fully appreciated through static images, the reader is strongly encouraged to watch the demonstration videos available in the project repository. Nevertheless, it is worth noting that the SLAM algorithm not only generates the occupancy grid map but also updates the costmaps used by the navigation stack. As a result, the robot is capable of performing fully autonomous navigation and frontier-based exploration.

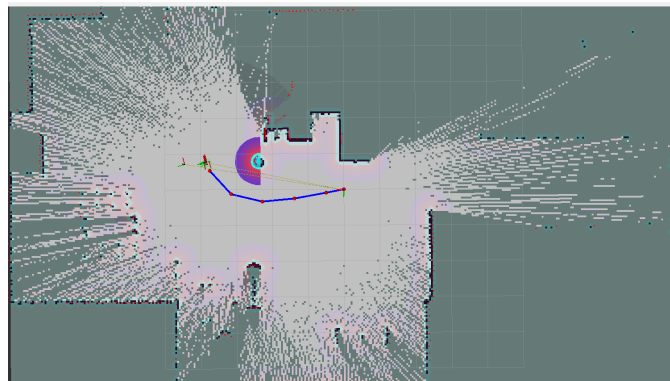


Figure 4.4: SLAM

Regarding the target-following capabilities, the robot is able to detect, track, and navigate towards an unknown dynamic target, provided that it is correctly identified. In

particular, the target must lie within a radius of approximately 3 meters from the robot and must be successfully distinguished from the static obstacles already represented in the occupancy grid.

This latter requirement is fundamental for the success of the proposed approach and strongly depends on the accuracy of the robot localization. A large localization uncertainty may lead to false positive detections, causing static obstacles to be incorrectly classified as dynamic targets and consequently tracked by the robot.

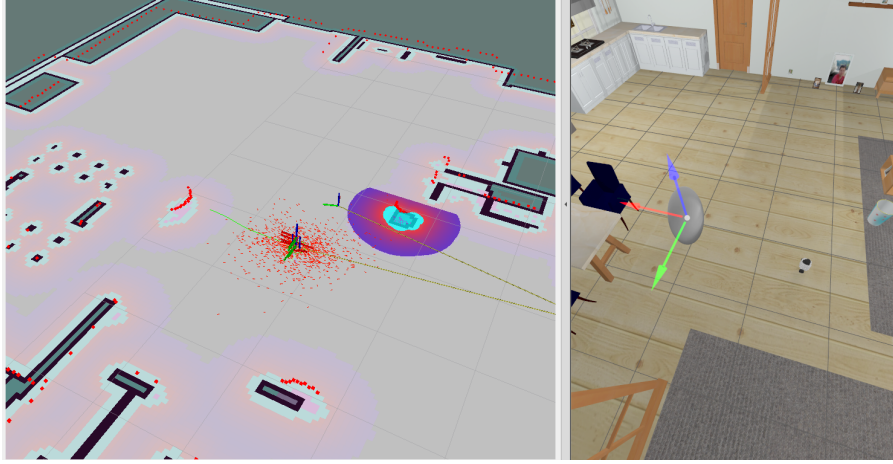


Figure 4.5: Target-following: a path leading to the dynamic target is being generated

## Chapter 5

# Critical discussion of results

The simulation results demonstrate that the proposed objectives have been successfully achieved. Thanks to the modular architecture of ROS 2 and the way the software has been designed, the real robot employs the same algorithms for localization, SLAM, navigation, and target following, requiring only minor modifications. Therefore, similar performance can reasonably be expected on the physical platform.

However, the current hardware implementation still presents two major limitations:

- Motion stuttering.
- Poor PID tuning.

The first issue becomes particularly evident when the target-following functionality is enabled. Under these conditions, a considerable number of `LaserScan` messages are dropped and prevents the local controller from operating at its desired frequency. This behavior suggests that the computational resources of the current hardware platform are insufficient to sustain the workload generated by the complete software stack.

The current PID tuning is not optimal, particularly when the robot's wheels are required to rotate at low speeds. This limitation negatively affects the smoothness and accuracy of the robot's motion. Nevertheless, improved performance can be achieved through a more accurate tuning of the PID controller parameters.